

Aufgabe "Qwirkle" (für Fortgeschrittene)

Schreiben Sie ein Java-Projekt, welches das Brett- und Legespiel „Qwirkle“ realisiert.

Qwirkle ist ein Strategiespiel für zwei bis vier Spieler. Die Spieler legen Spielsteine mit unterschiedlichen Farben und Symbolen auf einem gemeinsamen Spielfeld ab. Ziel des Spiels ist es, möglichst viele Punkte zu erzielen, indem Reihen gleicher Farbe oder gleichen Symbols gebildet werden.

Spielmaterial

Das Spiel besteht aus 108 Spielsteinen. Jeder Spielstein besitzt genau zwei Eigenschaften:

- eine Farbe
- ein Symbol
-

Es gibt sechs Farben:

- Rot
- Blau
- Grün
- Gelb
- Orange
- Lila

und sechs Symbole:

- Kreis
- Quadrat
- Raute
- Stern
- Kleeblatt
- Kreuz



Figure 1: Quelle: Wikipedia

Jede Kombination aus Farbe und Symbol kommt **dreimal** im Spiel vor.

Spielregeln

Für dieses Projekt gelten folgende Regeln:

- Das Spiel unterstützt 2 bis 4 Spieler.
- Zu Beginn erhält jeder Spieler sechs zufällige Spielsteine.
- Die übrigen Steine bilden den verdeckten Nachziehstapel.
- Es wird zufällig bestimmt, welcher Spieler beginnt.
- Während eines Zuges darf ein Spieler einen oder mehrere Spielsteine legen.

Alle neu gelegten Steine eines Zuges müssen

- in einer gemeinsamen Zeile oder Spalte liegen,
- unmittelbar an bereits vorhandene Steine angrenzen,
- eine gültige Qwirkle-Reihe bilden.

Gültige Reihen

Eine Reihe ist gültig, wenn alle Spielsteine die gleiche Farbe besitzen und unterschiedliche Symbole haben, oder alle Spielsteine das gleiche Symbol besitzen und unterschiedliche Farben haben.

Innerhalb einer Reihe darf

- kein Symbol doppelt vorkommen,
- keine Farbe doppelt vorkommen.

Eine Reihe darf höchstens sechs Spielsteine enthalten.

Qwirkle

Wird eine vollständige Reihe aus sechs Spielsteinen gebildet, erhält der Spieler einen Qwirkle-Bonus.

Punktewertung

Nach jedem Zug werden Punkte vergeben. Für jede neu entstandene oder erweiterte Reihe erhält der Spieler

- einen Punkt pro Spielstein der Reihe.

Wird dabei ein vollständiger Qwirkle erzeugt, erhält der Spieler zusätzlich 6 Bonuspunkte.

Die Punkte aller Reihen eines Zuges werden addiert.

Nachziehen

Nach Abschluss eines Zuges zieht der Spieler so viele Spielsteine nach, bis er wieder sechs Spielsteine besitzt oder der Nachziehstapel leer ist.

Spielende

Das Spiel endet,

- wenn keine Spielsteine mehr im Nachziehstapel vorhanden sind und
- ein Spieler seinen letzten Spielstein abgelegt hat.
-

Dieser Spieler erhält zusätzlich den Endspielbonus gemäß den Spielregeln.

Gewonnen hat der Spieler mit der höchsten Punktzahl.

Benutzeroberfläche

Implementieren Sie eine grafische Benutzeroberfläche. Diese soll mindestens folgende Funktionen besitzen:

- Darstellung des Spielfeldes
- Darstellung der Handkarten aller Spieler
- Anzeige des aktuellen Spielers
- Auswahl eines Spielsteins per Mausklick
- Platzieren eines Spielsteins auf dem Spielfeld
- Anzeige der aktuellen Punktestände
- Anzeige der verbleibenden Spielsteine im Nachziehstapel
- Anzeige des Spielgewinners

Spielablauf

Ein Spiel besteht aus folgenden Schritten:

1. Spielsteine mischen.
2. Spielsteine verteilen.
3. Startspieler bestimmen.
4. Spieler legt einen oder mehrere Spielsteine.
5. Gültigkeit des Zuges prüfen.
6. Punkte berechnen.
7. Spielsteine nachziehen.
8. Nächster Spieler.
9. Spielende feststellen.
10. Gewinner ausgeben.

Debug-Modus

Implementieren Sie einen Debug-Modus. Dieser soll mindestens folgende Funktionen besitzen:

- beliebige Handkarten erzeugen,
- beliebige Spielsteine auf dem Spielfeld platzieren,
- Nachziehstapel frei zusammenstellen,
- aktuelle Punktestände anzeigen,
- beliebigen Spieler als Startspieler festlegen.

Der Debug-Modus dient ausschließlich zum Testen während der Entwicklung und zur Vorstellung während der Prüfung.

Abnahmekriterien

- Objektorientierte Implementierung mit sinnvoller Klassenstruktur.
- Swing-GUI.
- Klare Trennung zwischen Spiellogik und Benutzeroberfläche.
- Geeignete Ausgaben und Statusanzeigen zur Demonstration des Spielablaufs.
- Das Spiel muss vollständig ohne Programmneustart spielbar sein.
- Clean Code (I): kurze Methoden (20–30 Zeilen, Ausnahmen sind zu begründen).
- Clean Code (II): maximale Einrückung pro Methode: 2 (3 nur mit Begründung).
- Clean Code (III): aussagekräftige Klassen-, Methoden- und Variablennamen.
- Erweiterbare Implementierung, sodass zusätzliche Spielregeln oder Varianten mit möglichst geringem Änderungsaufwand ergänzt werden können.

Zusatzaufgabe (optional)

Erweitern Sie das Programm um eine Zughilfe, die für den aktuellen Spieler alle regelkonformen Ablagemöglichkeiten bestimmt und auf Wunsch den Zug mit der höchsten erreichbaren Punktzahl hervorhebt. Die Spiellogik soll dabei unverändert bleiben und die Berechnung der Vorschläge in einer separaten Komponente erfolgen.